

Design Decisions

A small design decision I made a bit added on compared to the bootdev course was if the inputted txt file had 0 words, I also made the character count print 0 characters found, rather than just a characters subheading and then nothing underneath and straight to end. It just made the project seem completely thought through rather than only doing the necessities. I also fixed the grammar if there was only 1 word found, “from found 1 total words” to found 1 total word”, which once again made the project not feel rushed and or unfinished.

Test Cases

Test	Input	Expected Output	Actual Output	P/F
1	frankenstein.txt	Program displays word and character counts	<pre>===== BOOKBOT ===== Analyzing book found at books/frankenstein.txt... ----- Word Count ----- Found 75767 total words ----- Character Count ----- e: 44538 t: 29493 a: 25894 o: 24494 i: 23927 n: 23643 s: 20360 r: 20079 h: 19176 d: 16318 l: 12306 m: 10206 u: 10111 c: 9011 f: 8451 y: 7756 w: 7450 p: 5952 g: 5795 b: 4868 v: 3737 k: 1661 x: 691 j: 497 q: 325 z: 235 æ: 28 â: 8 ê: 7 ë: 2 ô: 1 ===== END =====</pre>	Pass
2	mobydick.txt	Program displays word and character	<pre>===== BOOKBOT ===== Analyzing book found at books/mobydick.txt... ----- Word Count ----- Found 215838 total words</pre>	Pass

		counts	<pre> ----- Character Count ----- e: 119354 t: 89875 a: 79224 o: 70809 n: 66781 i: 66675 s: 65139 h: 63769 r: 53593 l: 43351 d: 38840 u: 27204 m: 23627 c: 23319 w: 22557 g: 21287 f: 21252 p: 17874 y: 17243 b: 17204 v: 8725 k: 8228 q: 1581 j: 1177 x: 1064 z: 636 æ: 23 œ: 5 é: 5 è: 3 l :n l :r x: 1 n: 1 r: 1 o: 1 ç: 1 â: 1 ===== END ===== </pre>	
3	prideandprejudice.txt	Program displays word and character counts	<pre> ===== BOOKBOT ===== Analyzing book found at books/prideandprejudice.txt... ----- Word Count ----- Found 130410 total words ----- Character Count ----- e: 74451 t: 50837 a: 44834 o: 43383 i: 41198 n: 40686 h: 36162 s: 35695 r: 35168 d: 23723 l: 23475 u: 16303 m: 15676 c: 14838 y: 13579 w: 13017 f: 12996 g: 11007 b: 9762 p: 9154 v: 6118 k: 3497 x: 1032 j: 1014 z: 971 q: 660 é: 8 â: 4 è: 2 â: 1 œ: 1 </pre>	Pass

			===== END =====	
4	Empty/invalid argument	Program displays an appropriate error/usage message	Usage: python3 main.py <path_to_book>	Pass
5	A file containing a single word	Program correctly counts one word	===== BOOKBOT ===== Analyzing book found at books/oneword.txt... ----- Word Count ----- Found 1 total word ----- Character Count ----- l: 2 h: 1 e: 1 o: 1 ===== END =====	Pass
6	A file containing no words	Program correctly identifies 0 words counted	===== BOOKBOT ===== Analyzing book found at books/nowords.txt... ----- Word Count ----- Found 0 total words ----- Character Count ----- Found 0 total characters ===== END =====	Pass

Challenge + Solution

One challenge I experienced was with my initial `char_count_dict`. I originally had every letter of the alphabet already stored as a key, but this was highly inefficient because it created entries for characters that did not appear in the book. I changed the way the dictionary was created so that characters were only added when they appeared in the text. This made the program more efficient and helped me understand how dictionaries can be used to store data dynamically.